Introduction to Machine Learning

Mihir Rao

Contents

1 Basics **2**
- 1.1 Training data and hypotheses . . . 2
- 1.2 Linear classifiers and their parameters . . . 3
- 1.3 Why the dot product describes the boundary . . . 4
- 1.4 Loss: measuring a prediction's cost . . . 5
- 1.5 Training error and test error . . . 5
- 1.6 Learning a classifier . . . 6

2 Perceptrons **8**
- 2.1 The classification setup . . . 8
- 2.2 The perceptron algorithm . . . 8
- 2.3 What one update accomplishes . . . 9
- 2.4 Linear separability . . . 10
- 2.5 Signed distance and margin . . . 10
- 2.6 The perceptron performance theorem . . . 12
- 2.7 Including an offset by expanding the features . . . 13
- 2.8 When the data are not linearly separable . . . 13
- 2.9 Where classification fits among learning tasks . . . 14

3 Features **15**
- 3.1 From a goal to a learning analysis . . . 15
- 3.2 Encoding categories for a linear classifier . . . 16
- 3.3 Encoding information with several components . . . 17
- 3.4 Ranges and ordered categories . . . 18
- 3.5 Numerical scales and standardization . . . 18
- 3.6 Nonlinear boundaries from new features . . . 20
- 3.7 Polynomial features and flexible boundaries . . . 21
- 3.8 Evaluation with held-out data . . . 22
- 3.9 Cross-validation . . . 23

Lecture 1: Basics

Machine learning uses data to choose prediction rules. Supervised learning predicts labels for new examples from labeled training data.

Given examples with known labels, how can we label a new example? We need data, possible prediction rules, a measure of mistakes, and an algorithm to choose a rule.

1.1 Training data and hypotheses

Definition 1.1 [Labeled training data]. For each $i \in \{1, \dots, n\}$, the feature vector is

$$x^{(i)} = \left(x_1^{(i)}, \dots, x_d^{(i)}\right)^\mathsf{T} \in \mathbb{R}^d,$$

and its label is $y^{(i)} \in \{-1, +1\}$. The training data consist of the n labeled examples

$$\mathcal{D}_n = \left\{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\right\}.$$

The superscript (i) identifies the example; the subscript identifies a feature. Thus $x_2^{(3)}$ is the second feature of the third example, rather than a power of x_2. The transpose makes each feature vector a column vector.

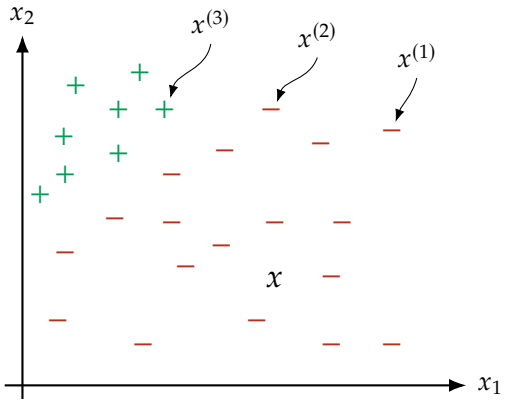


Figure 1: Schematic labeled data in two dimensions. Each position is a feature vector; its + or − gives the observed label. The point marked x still needs a prediction. The coordinates illustrate structure, not measurements.

Definition 1.2 [Hypothesis]. A hypothesis, or prediction rule, is a function

$$h : \mathbb{R}^d \longrightarrow \{-1, +1\}.$$

Given a feature vector x, it returns a predicted label $h(x)$. In this binary labeling problem, such a rule is also called a CLASSIFIER.

Example: *Always predict* +1. The rule $h(x) = +1$ for every x is a valid hypothesis: it assigns an allowed label to every input. Being a hypothesis does not make it a good one. This rule gets every example with label -1 wrong.

The task is to predict labels for new feature vectors, rather than merely to describe labels already present in $\mathcal{D}_n$.

1.2 Linear classifiers and their parameters

We need to decide which prediction rules the learning algorithm may consider. A HYPOTHESIS CLASS $\mathcal{H}$ is a set of hypotheses. A useful first class assigns +1 to one side of a line and -1 to the other. In d dimensions, the corresponding boundary is a hyperplane.

Definition 1.3 [LINEAR CLASSIFIER]. Let $\theta \in \mathbb{R}^d$ and $\theta_0 \in \mathbb{R}$. A LINEAR CLASSIFIER IS

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\mathsf{T} x + \theta_0) = \begin{cases} +1, & \theta^\mathsf{T} x + \theta_0 > 0, \\ -1, & \theta^\mathsf{T} x + \theta_0 \leq 0. \end{cases}$$

Here the sign convention assigns -1 when the score is exactly zero. The PARAMETERS are the vector θ and the scalar θ_0. The class $\mathcal{H}$ contains all rules of this form.

The score $\theta^\mathsf{T} x + \theta_0$ decides the label. When $\theta \neq 0$, the DECISION BOUNDARY IS

$$\{x : \theta^\mathsf{T} x + \theta_0 = 0\}.$$

The vector θ sets the boundary's orientation and points toward the positive side. The scalar θ_0 sets its offset. For $\theta = 0$, the score is constant, so the rule predicts one label everywhere; there is no ordinary separating hyperplane. The geometric formulas below assume $\theta \neq 0$.

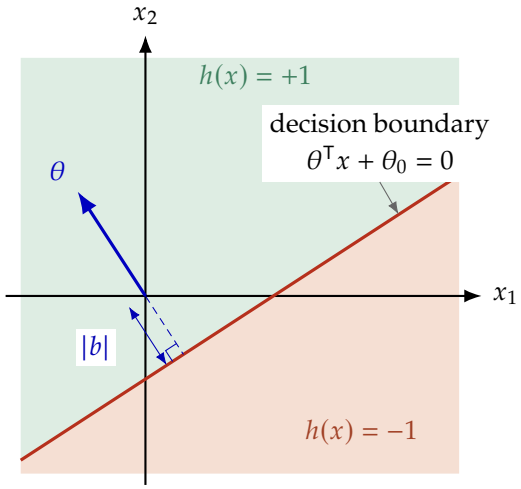


Figure 2: A schematic linear classifier. The normal θ points toward the positive region. The boundary is included in the negative region by the stated sign convention. Its offset is measured along the unit normal.

1.3 Why the dot product describes the boundary

The quantity $\theta^\mathsf{T} x$ measures how far x extends in the direction of θ, up to a scale factor. Write

$$u = \frac{\theta}{\|\theta\|}, \qquad \|\theta\| = \sqrt{\theta^\mathsf{T}\theta}.$$

Since u has length one, the SIGNED PROJECTION of x along this direction is

$$u^\mathsf{T} x = \frac{\theta^\mathsf{T} x}{\|\theta\|}.$$

Positive values are in the direction of θ; negative values are in the opposite direction. Dividing the boundary equation by $\|\theta\|$ gives

$$\frac{\theta^\mathsf{T} x}{\|\theta\|} = -\frac{\theta_0}{\|\theta\|}.$$

Thus every point on the boundary has the same signed projection. If b denotes that projection, then

$$b = -\frac{\theta_0}{\|\theta\|}, \qquad \theta^\mathsf{T} x + \theta_0 > 0 \iff u^\mathsf{T} x > b.$$

The positive region lies beyond this threshold in the direction of θ. The perpendicular distance from the origin to the boundary is $|b|$; b itself is signed.

For two boundary points p and q, subtract their equations:

$$(\theta^\mathsf{T} p + \theta_0) - (\theta^\mathsf{T} q + \theta_0) = 0 \quad \Longrightarrow \quad \theta^\mathsf{T}(p - q) = 0.$$

Since $p - q$ runs along the boundary, its zero dot product with θ establishes perpendicularity.

1.4 Loss: measuring a prediction's cost

A classifier should work well on future data. Before comparing classifiers, we must specify what a mistake costs.

Definition 1.4 [Loss]. A LOSS $L(g, a)$ gives the cost of a guessed label g when the actual label is a. For example, the ZERO-ONE LOSS is

$$L(g, a) = \begin{cases} 0, & g = a, \\ 1, & g \neq a. \end{cases}$$

Zero-one loss treats the two kinds of wrong prediction equally. A different task may require different penalties.

Example: *Asymmetric mistakes.* Consider

$$L(g, a) = \begin{cases} 1, & g = +1,\ a = -1, \\ 100, & g = -1,\ a = +1, \\ 0, & g = a. \end{cases}$$

Predicting -1 when the actual label is $+1$ costs one hundred times as much as the reverse mistake. A correct prediction costs zero in either class.

The order of the arguments matters: the first is the guess and the second is the actual label. With asymmetric loss, exchanging them changes the cost.

1.5 Training error and test error

To evaluate a classifier on several examples, average its losses. The TRAINING ERROR on the data used for learning is

$$\mathcal{E}_n(h) = \frac{1}{n} \sum_{i=1}^{n} L\big(h(x^{(i)}), y^{(i)}\big).$$

With zero-one loss, the sum counts misclassified examples, so $\mathcal{E}_n(h)$ is the fraction of training examples predicted incorrectly. With asymmetric loss, it is an average cost, rather than a fraction of mistakes.

The TEST ERROR on n' new examples, indexed $n+1, \ldots, n+n'$, is

$$\mathcal{E}(h) = \frac{1}{n'} \sum_{i=n+1}^{n+n'} L\big(h(x^{(i)}), y^{(i)}\big).$$

The two formulas use the same loss but different data. The test average measures performance on the new examples, which is the behavior we want to assess.

When choosing from training data, a natural preference is h over $\widetilde{h}$ if $\mathcal{E}_n(h) < \mathcal{E}_n(\widetilde{h})$. This compares their training performance. It does not establish the same ordering of their test errors.

1.6 Learning a classifier

A classifier takes an input vector and returns a label. A LEARNING ALGORITHM has a different input and output: it takes training data and returns a classifier from the chosen class. For linear classifiers, this means choosing θ and θ_0 using $\mathcal{D}_n$.

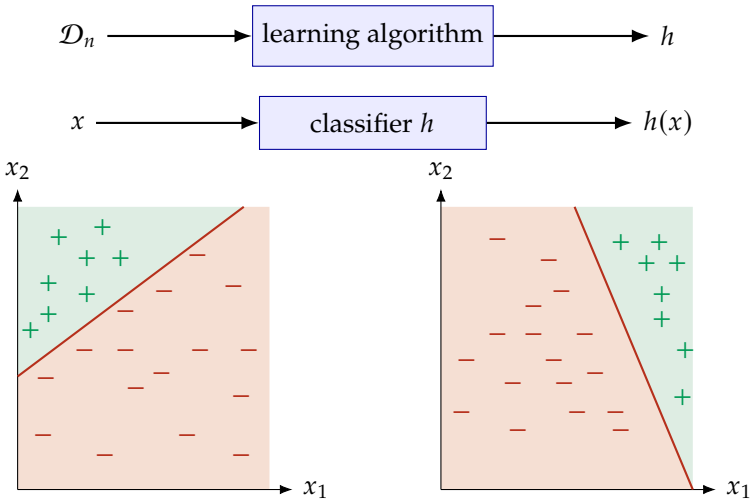


Figure 3: Learning chooses a rule from labeled data; prediction applies that rule to a feature vector. The two schematic data arrangements below admit different separating lines, illustrating why the chosen parameters depend on the training data.

An example: search a random candidate pool

One conceptual learning algorithm constructs many candidate classifiers and selects the one with the smallest training error. Let $M = 10^{12}$, and randomly sample a fixed pool of M parameter pairs. For an integer $1 \le k < M$, compare only the first k candidates. Write

$$h^{(j)}(x) = h\big(x; \theta^{(j)}, \theta_0^{(j)}\big), \qquad j = 1, \ldots, M.$$

The superscript (j) now indexes candidates, while (i) indexes data points.

Algorithm 1.1: Select a classifier from a random candidate pool

Input: Training data $\mathcal{D}_n$; integer $1 \le k < M$, where $M = 10^{12}$.
Output: A classifier with smallest training error among the first k candidates.

for $j \leftarrow 1$ **to** M **do**
  sample a parameter pair $(\theta^{(j)}, \theta_0^{(j)})$
  Set $h^{(j)}(x) \leftarrow h(x; \theta^{(j)}, \theta_0^{(j)})$
$j^* \leftarrow \arg\min_{j \in \{1,\dots,k\}} \mathcal{E}_n(h^{(j)})$
return $h^{(j^*)}$

If several candidates have the same minimum error, choose the smallest index. The data determine which candidate wins within the pool. The number k is a HYPERPARAMETER: it controls the learning procedure, rather than appearing in the classifier's prediction formula. The classifier's parameters remain θ and θ_0.

Proposition 1.5 [COMPARING ONE CANDIDATE WITH TWO]. *For the same data, loss, and fixed candidate pool, the rule selected with $k = 2$ has training error no larger than the rule selected with $k = 1$.*

Proof. The first candidate remains available when the second is added. With $k = 1$, the selected error is $\mathcal{E}_n(h^{(1)})$. With $k = 2$, it is

$$\min\{\mathcal{E}_n(h^{(1)}), \mathcal{E}_n(h^{(2)})\} \le \mathcal{E}_n(h^{(1)}).$$

It decreases if the second candidate has strictly smaller error; otherwise it stays the same. □

This comparison requires a shared pool, so the first candidate is the same in both runs. It does not compare independent random draws and does not guarantee an improvement in test error.

Source. Tamara Broderick, *Introduction to Machine Learning*, MIT 6.036/6.862, Lecture 1, Fall 2020.

Lecture 2: Perceptrons

The perceptron starts at zero and updates parameters when an example fails strict separation. Positive-margin separability bounds the number of updates.

2.1 The classification setup

Our data are $\mathcal{D}_n = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$, with $x^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{-1, +1\}$. A linear classifier uses

$$h(x; \theta, \theta_0) = \begin{cases} +1, & \theta^\top x + \theta_0 > 0, \\ -1, & \theta^\top x + \theta_0 \le 0. \end{cases}$$

The class $\mathcal{H}$ consists of these rules as θ and θ_0 vary. For $\theta \neq 0$, the boundary $\theta^\top x + \theta_0 = 0$ is a hyperplane with normal vector θ.

With zero-one loss, $L(g, a) = 0$ if $g = a$ and 1 otherwise. Thus

$$\mathcal{E}_n(h) = \frac{1}{n} \sum_{i=1}^{n} L\big(h(x^{(i)}), y^{(i)}\big)$$

is the fraction of training examples misclassified. Our aim is to find parameters that make this error small.

2.2 The perceptron algorithm

For an example $(x^{(i)}, y^{(i)})$, multiply the classifier's score by its label:

$$y^{(i)}\big(\theta^\top x^{(i)} + \theta_0\big).$$

This label-weighted score is positive when the example is strictly on the side prescribed by its label, negative when it is on the wrong side, and zero when it lies on the boundary. A negative label reverses the score's sign, so the same test works for both classes.

The perceptron updates whenever this quantity is nonpositive:

$$\theta \leftarrow \theta + y^{(i)} x^{(i)}, \qquad \theta_0 \leftarrow \theta_0 + y^{(i)}.$$

Positive examples add $x^{(i)}$ and 1; negative examples subtract $x^{(i)}$ and 1.

Algorithm 2.1: Perceptron with an offset

Input: Labeled data $\mathcal{D}_n$; maximum number of passes $\tau \geq 1$.
Output: Parameters (θ, θ_0) of a linear classifier.

$\theta \leftarrow \mathbf{0} \in \mathbb{R}^d$; $\theta_0 \leftarrow 0$
for $t \leftarrow 1$ **to** τ **do**
  changed $\leftarrow$ false
  for $i \leftarrow 1$ **to** n **do**
    if $y^{(i)}(\theta^\mathsf{T} x^{(i)} + \theta_0) \leq 0$ **then**
      $\theta \leftarrow \theta + y^{(i)} x^{(i)}$
      $\theta_0 \leftarrow \theta_0 + y^{(i)}$
      changed $\leftarrow$ true
  if changed = false **then**
    break
return (θ, θ_0)

The initial vector has d zeros, one for each feature. Each PASS visits all n examples, using the current parameters after any earlier update. The pass budget τ is a hyperparameter of the learning procedure.

Note. The update test includes *every* boundary point. With our sign convention, a boundary point labeled -1 is already predicted correctly, but it still triggers an update because its label-weighted score is zero. Initially every score is zero, so the first visited example triggers an update as well.

If a whole pass makes no changes, the parameters remain fixed throughout it and every example has a positive label-weighted score. All training examples are then classified correctly. Reaching the cap τ is a different stopping condition: it does not by itself establish zero training error.

2.3 What one update accomplishes

An update increases the label-weighted score of the example that caused it. To see this, abbreviate that example to (x, y) and let $\theta' = \theta + yx$ and $\theta'_0 = \theta_0 + y$. Expanding the new score gives

$$\begin{aligned} y\big((\theta')^\mathsf{T} x + \theta'_0\big) &= y\big((\theta + yx)^\mathsf{T} x + \theta_0 + y\big) \\ &= y(\theta^\mathsf{T} x + \theta_0) + y^2(x^\mathsf{T} x + 1) \\ &= y(\theta^\mathsf{T} x + \theta_0) + \|x\|^2 + 1. \end{aligned}$$

The last step uses $y \in \{-1, +1\}$, hence $y^2 = 1$, and $x^\mathsf{T} x = \|x\|^2$. The $+1$ comes from updating the offset. Since $\|x\|^2 + 1 > 0$, the score strictly increases.

The score can still be nonpositive, and other examples can remain wrong. Repeated scans are therefore needed.

2.4 Linear separability

Definition 2.1 [Linear separability]. A training set is linearly separable if there exist θ and θ_0 such that

$$y^{(i)}\left(\theta^{\mathsf{T}} x^{(i)} + \theta_0\right) > 0 \qquad \text{for every } i \in \{1, \ldots, n\}.$$

Both classes must be strictly on their respective sides of the same boundary. Positive examples have positive scores; negative examples have negative scores. No training point is allowed on the boundary in this definition.

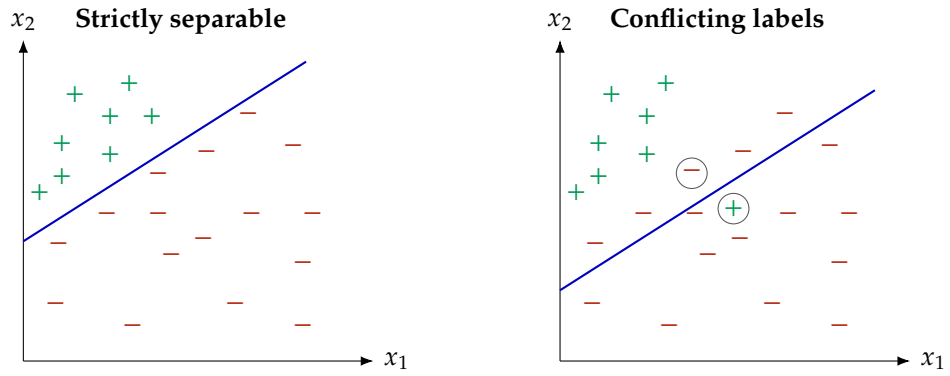


Figure 4: Schematic redraws of two labeled arrangements. The left set admits strict separation. The right set contains conflicting points among the opposite class; reversing the displayed line's orientation does not solve the separation problem. Circles identify the conflicting points.

2.5 Signed distance and margin

Correct separation tells us which side a point occupies. Distance tells us how far it is from the boundary, which lets us distinguish easier separation from separation with points very close to the boundary.

For $\theta \neq 0$, the signed projection of x^* along the unit normal is $\theta^{\mathsf{T}} x^* / \|\theta\|$. The boundary's signed projection from the origin is $b = -\theta_0 / \|\theta\|$. Subtracting these gives the signed distance from the boundary to the point:

$$\delta(x^*; \theta, \theta_0) = \frac{\theta^{\mathsf{T}} x^*}{\|\theta\|} - \left(-\frac{\theta_0}{\|\theta\|}\right) = \frac{\theta^{\mathsf{T}} x^* + \theta_0}{\|\theta\|}.$$

It is positive on the positive side, negative on the negative side, and zero on the boundary. Its absolute value is the perpendicular distance.

Definition 2.2 [Point and training-set margins]. The margin of a labeled point (x^*, y^*) with respect to a hyperplane with $\theta \neq 0$ is

$$m(x^*, y^*; \theta, \theta_0) = y^* \frac{\theta^\mathsf{T} x^* + \theta_0}{\|\theta\|}.$$

The margin of the training set with respect to that hyperplane is the smallest point margin:

$$m(\mathcal{D}_n; \theta, \theta_0) = \min_{i \in \{1,\ldots,n\}} y^{(i)} \frac{\theta^\mathsf{T} x^{(i)} + \theta_0}{\|\theta\|}.$$

Multiplying by the label makes a correctly separated point's margin positive in either class. A negative-class point can have negative signed distance and positive margin. A point on the wrong side has negative margin; a boundary point has zero margin. The training-set margin checks the least favorable example, so it is positive precisely when this hyperplane strictly separates the whole set.

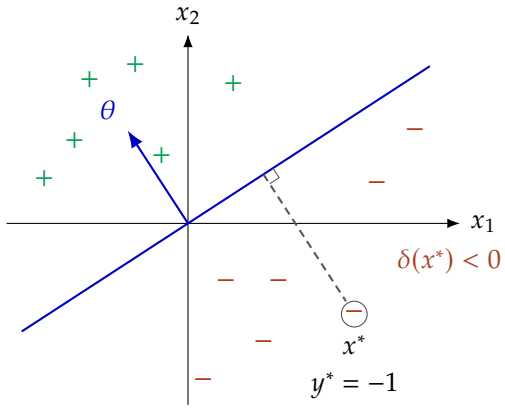


Figure 5: A schematic negative-class point has $\delta(x^*) < 0$ but positive margin $y^*\delta(x^*)$ when $y^* = -1$. The dashed segment is perpendicular to the boundary, rather than a horizontal or vertical distance.

2.6 The perceptron performance theorem

A strictly separating hyperplane with a positive margin gives a bound on the number of perceptron updates. Here we first restrict the classifier to pass through the origin: hold $\theta_0 = 0$ throughout and update only θ.

Theorem 2.3 [Perceptron performance]. *Consider the origin-constrained perceptron, initialized at $\theta = 0$. Assume there exist a nonzero separator θ^* and a number $\gamma > 0$ such that, for every training example,*

$$y^{(i)} \frac{(\theta^*)^\mathsf{T} x^{(i)}}{\|\theta^*\|} > \gamma.$$

Also assume there is an R such that $\|x^{(i)}\| \le R$ for every i. Then the algorithm makes at most

$$\left(\frac{R}{\gamma}\right)^2$$

updates to θ. A complete pass without an update has zero training error.

The vector θ^* is a fixed separating rule whose existence is assumed; the algorithm does not need to know it. The number γ is a positive lower bound on its point margins. The radius R bounds how far any feature vector lies from the origin.

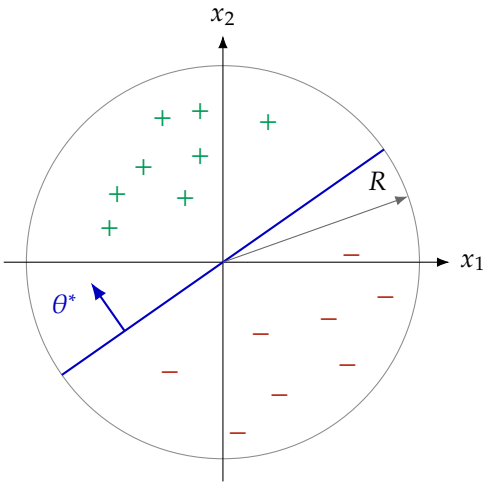


Figure 6: Schematic theorem assumptions: the separating line passes through the origin, all points lie within radius R, and their label-adjusted distances from the line exceed a positive lower bound γ.

The bound counts *updates*, not passes or point visits. A smaller guaranteed margin increases the bound. It guarantees eventual termination on data satisfying the assumptions when enough passes are allowed, but does not promise completion before an arbitrarily small pass cap.

2.7 Including an offset by expanding the features

Restricting a hyperplane to the origin need not restrict the original prediction rule. Add one constant feature and put the offset into its weight:

$$x_{\text{new}} = \begin{bmatrix} x_1 & \cdots & x_d & 1 \end{bmatrix}^{\mathsf{T}}, \qquad \theta_{\text{new}} = \begin{bmatrix} \theta_1 & \cdots & \theta_d & \theta_0 \end{bmatrix}^{\mathsf{T}}.$$

Both vectors are in $\mathbb{R}^{d+1}$, and

$$\theta_{\text{new}}^{\mathsf{T}} x_{\text{new}} = \theta^{\mathsf{T}} x + \theta_0.$$

The scores and predicted labels are unchanged. An offset boundary in the original space becomes a boundary through the origin in the expanded space.

The expanded perceptron update also recovers both original updates:

$$\theta_{\text{new}} \leftarrow \theta_{\text{new}} + y\, x_{\text{new}} \quad \Longleftrightarrow \quad \begin{cases} \theta \leftarrow \theta + yx, \\ \theta_0 \leftarrow \theta_0 + y. \end{cases}$$

Apply the theorem using the radius and margins in the expanded space. In particular, $\|x_{\text{new}}\| = \sqrt{\|x\|^2 + 1}$, and the margin uses $\|\theta_{\text{new}}\| = \sqrt{\|\theta\|^2 + \theta_0^2}$ in its denominator. The original radius and margin therefore cannot simply be reused unchanged.

2.8 When the data are not linearly separable

Real data need not admit strict linear separation. If no separating rule exists in the chosen class, the theorem's positive-margin assumption fails. Repeated perceptron updates then have no convergence guarantee from this theorem, and the pass cap still provides a stopping condition.

Example: *Penguin measurements.* Flipper length and body mass form a two-dimensional feature vector; species supplies a label. The Adelie and Chinstrap groups overlap in these features. A perfect straight boundary should not be assumed merely because the examples have labels.

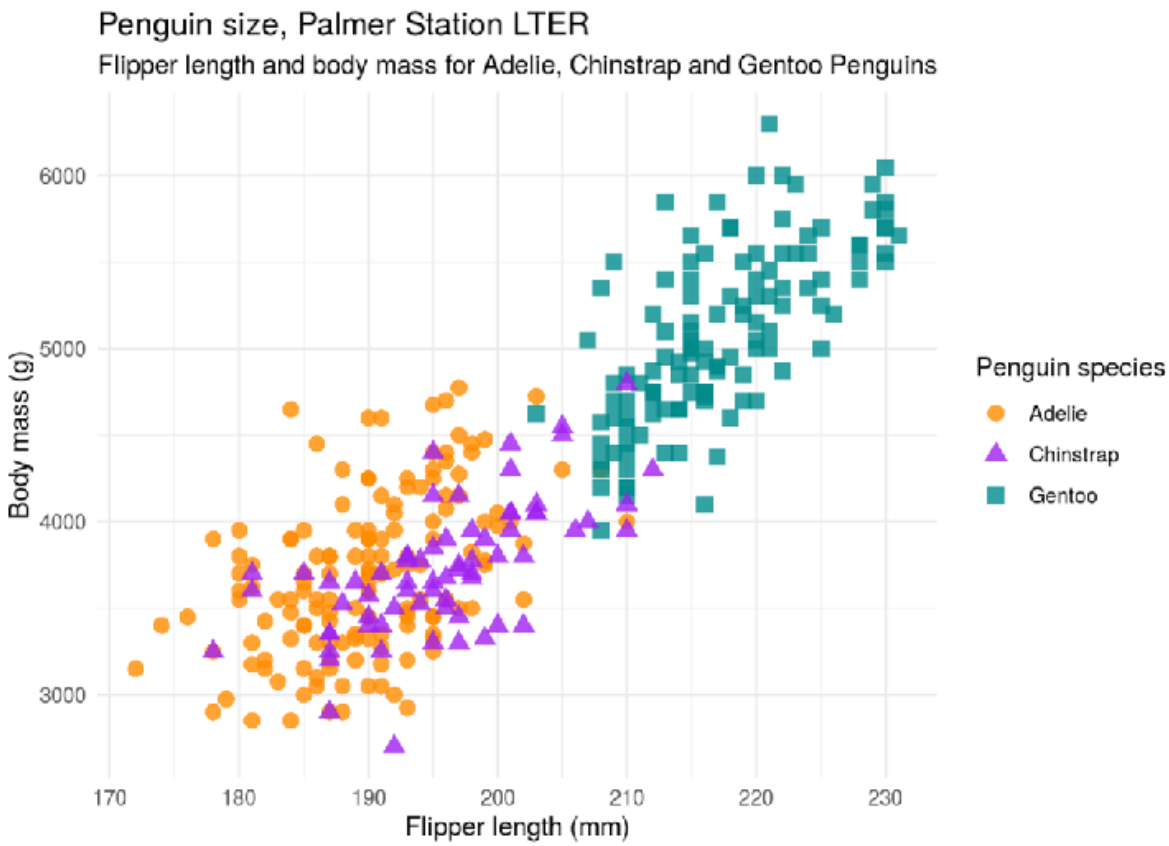


Figure 7: Penguin flipper length and body mass, colored by species. The overlapping groups show why strict linear separation must be checked.

2.9 Where classification fits among learning tasks

Definition: Supervised learning uses labeled examples to learn a mapping from features to labels. Unsupervised learning has no supplied labels and seeks patterns in the data.

Within supervised learning, the output type distinguishes two tasks. Regression predicts continuous values, modeled as a mapping $\mathbb{R}^d \to \mathbb{R}^k$. Classification predicts a value in a discrete label set. Binary classification, or two-class classification, uses two labels, such as $\{-1, +1\}$. Multiclass classification uses more than two label values. Linear classification is one choice of hypothesis class for the binary task.

Source. Tamara Broderick, *Introduction to Machine Learning*, MIT 6.036/6.862, Lecture 2, Fall 2020. Penguin data: Palmer Penguins.

Lecture 3: Features

Feature choices determine which patterns a linear classifier can express; held-out data evaluate whether the learned rule predicts new observations.

3.1 From a goal to a learning analysis

A complete analysis has four parts: establish a goal and obtain data, encode the data, run a learning algorithm, and interpret and evaluate its output. For binary classification, the output remains a rule with score $\theta^\top x + \theta_0$, zero-one loss measures mistakes, and training error averages that loss over the training set. A finite candidate search, perceptron, or averaged perceptron is a possible learning procedure.

Example: *Heart-disease classification.* The goal is to predict whether someone has heart disease from the information available about them. The following four observations contain several kinds of information, not yet one uniform numerical vector.

	Disease?	Heart rate (bpm)	Pain?	Job	Medicines	Age	Income (USD)
1	no	55	no	nurse	pain	40s	133000
2	no	71	no	admin	beta blockers, pain	20s	34000
3	yes	89	yes	nurse	beta blockers	50s	40000
4	no	67	no	doctor	none	50s	120000

Table 1: The illustrative observations used throughout the encoding examples.

First identify the target. Encode disease as yes $\mapsto +1$ and no $\mapsto -1$, giving $(y^{(1)}, \ldots, y^{(4)}) = (-1, -1, +1, -1)$. Some algorithms instead expect $\{0, 1\}$. Save the chosen mapping so numerical predictions can be translated back into their original meanings.

Definition 3.1 [Feature map]. A feature is a function of the input information, excluding the target label. A feature map ϕ collects chosen numerical features into a vector $\phi(x)$. We write x for the old representation and $\phi(x)$ for the new one.

The target column supplies y, not an input feature. Heart rate is already numerical; pain can be encoded as no $\mapsto 0$, yes $\mapsto 1$. Occupation, medicines, and age ranges require further choices.

3.2 Encoding categories for a linear classifier

Categorical data have distinct values without a meaningful order. Occupation is an example. Assigning each occupation a unique integer preserves its identity, but also introduces an order and numerical spacing that the original categories do not have.

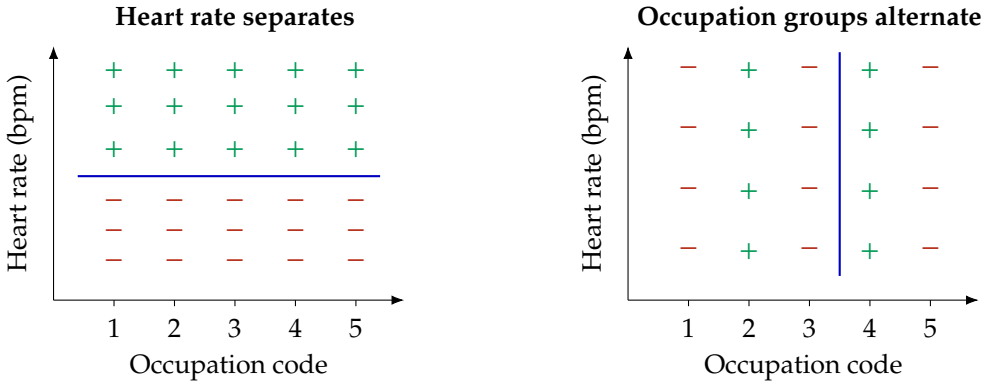


Figure 8: Schematic occupation and heart-rate arrangements. A horizontal line separates the left arrangement. On the right, doctors and pharmacists are positive while the other occupations are negative. One straight boundary cannot alternate between these groups at the same heart rate. The displayed candidate line illustrates the mismatch.

A compact alternative assigns a distinct binary code to each category. It still imposes relationships: categories sharing a bit share that feature. The following three encodings use the same five occupations.

Occupation	Integer	Binary code	One-hot vector
nurse	1	$(0, 0, 0)$	$(1, 0, 0, 0, 0)$
admin	3	$(0, 0, 1)$	$(0, 1, 0, 0, 0)$
pharmacist	4	$(0, 1, 0)$	$(0, 0, 1, 0, 0)$
doctor	2	$(0, 1, 1)$	$(0, 0, 0, 1, 0)$
social worker	5	$(1, 0, 0)$	$(0, 0, 0, 0, 1)$

Table 2: The integer, binary, and one-hot encodings of occupation. Their coordinate order is part of the saved encoding.

For the first four binary codes, the first bit is zero. Plot the remaining bits as b_2, b_3 to see which label patterns a linear rule can separate.

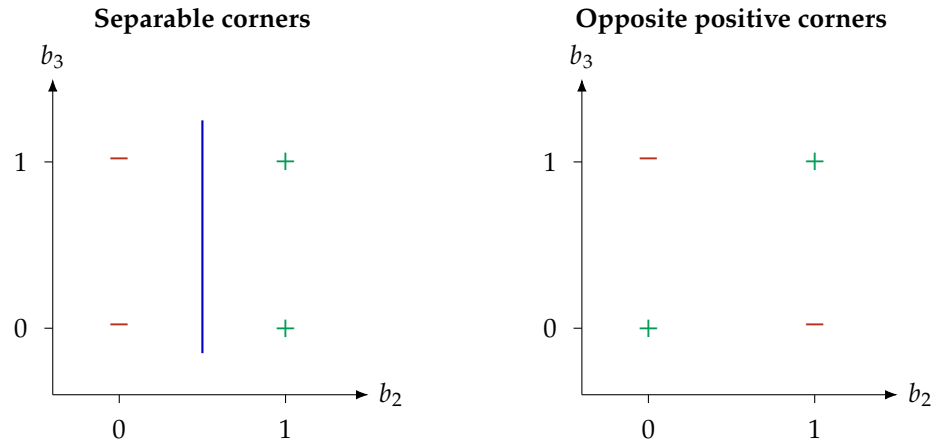


Figure 9: Schematic labels on the four binary-code corners. The left pattern is separated by $b_2 = 1/2$. The right pattern has positive opposite corners; no straight line strictly separates it. Distinct codes alone do not ensure that a linear classifier can express the desired categories.

To check the right pattern, write its score as $ab_2 + bb_3 + c$. Positivity at $(0, 0)$ and $(1, 1)$ requires $c > 0$ and $a + b + c > 0$. Negative predictions at $(1, 0)$ and $(0, 1)$ require $a + c \leq 0$ and $b + c \leq 0$. Adding the latter gives $a + b + 2c \leq 0$, whereas adding the former gives $a + b + 2c > 0$: a contradiction.

Definition: One-hot encoding gives each category its own indicator coordinate. Exactly one coordinate is 1 and the others are 0.

One-hot encoding gives category j its own coefficient θ_j, without imposing a ranking or shared bit weights. The job coordinates $j_1, \ldots, j_5$ follow the table order.

3.3 Encoding information with several components

Medicines can occur together. We could give each of four combinations its own one-hot coordinate, but that hides the fact that a combination contains two separately meaningful components.

Definition: Factored encoding represents those components separately. Let m_1 indicate pain medication and m_2 indicate beta blockers.

Medicines	One-hot vector	Factored (m_1, m_2)
pain	$(1, 0, 0, 0)$	$(1, 0)$
pain and beta blockers	$(0, 1, 0, 0)$	$(1, 1)$
beta blockers	$(0, 0, 1, 0)$	$(0, 1)$
none	$(0, 0, 0, 1)$	$(0, 0)$

Table 3: Combination categories versus separate medication indicators.

Both encodings distinguish the four combinations. The factored version also records shared

components directly, and uses two coordinates instead of four.

3.4 Ranges and ordered categories

An age given as "40s" is a range, not an exact observation. Replacing it by 45 gives a representative value, but can create false precision: a later reader may treat 45 as the person's actual age. The four age ranges become $45, 25, 55, 55$ under this choice. Using decades instead gives $4, 2, 5, 5$ and keeps the recorded level of detail visible. Plotting the data helps reveal problems caused by the representation.

Definition: Numerical data have meaningful ordering and differences in value. Ordinal data have meaningful ordering, but their numerical gaps need not have meaning. Categorical values have no inherent ordering.

A Likert response runs from strongly disagree to strongly agree. Codes $1, 2, 3, 4, 5$ preserve that order, but equal numerical gaps are an extra choice. Stretching the gap before the last response can change the geometry seen by a linear classifier even though the response order is unchanged.

Definition: A unary encoding, also called a thermometer code, turns on coordinates cumulatively as the ordered level increases.

Response	Level	Thermometer code
strongly disagree	1	$(1, 0, 0, 0, 0)$
disagree	2	$(1, 1, 0, 0, 0)$
neutral	3	$(1, 1, 1, 0, 0)$
agree	4	$(1, 1, 1, 1, 0)$
strongly agree	5	$(1, 1, 1, 1, 1)$

Table 4: The cumulative five-coordinate encoding of ordered responses.

For level r, coordinate ℓ is $\mathbf{1}\{r \geq \ell\}$ for $\ell = 1, \ldots, 5$. A linear score adds another coordinate's weight when the level rises, allowing adjacent levels to have different score increments.

3.5 Numerical scales and standardization

A plot can show which numerical features a fitted rule uses. In a schematic plot of heart rate against weekly garlic consumption, a horizontal boundary uses heart rate alone: moving horizontally does not change the prediction. With income and heart rate, different numerical scales can make the same configuration look broad, narrow, or almost vertical.

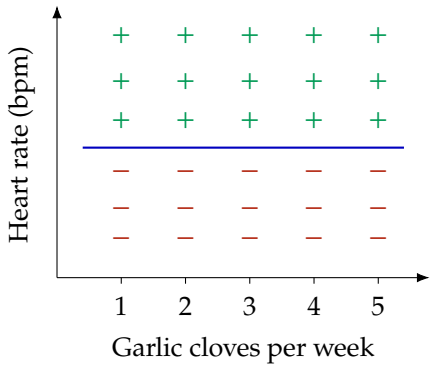


Figure 10: Schematic heart rate and garlic-consumption example. The horizontal boundary ignores the horizontal feature. No measured heart rates are implied.

Definition 3.2 [Standardization]. For numerical coordinate j, subtract its mean μ_j and divide by its positive standard deviation s_j:

$$\phi_j(x^{(i)}) = \frac{x_j^{(i)} - \mu_j}{s_j}.$$

This standardization centers the feature and expresses differences in units of its standard deviation.

A value equal to the mean becomes 0; one standard deviation above the mean becomes 1. A constant coordinate has $s_j = 0$, so this formula cannot be applied to it unchanged. Standardization changes numerical scales; it does not turn categories into meaningful measurements.

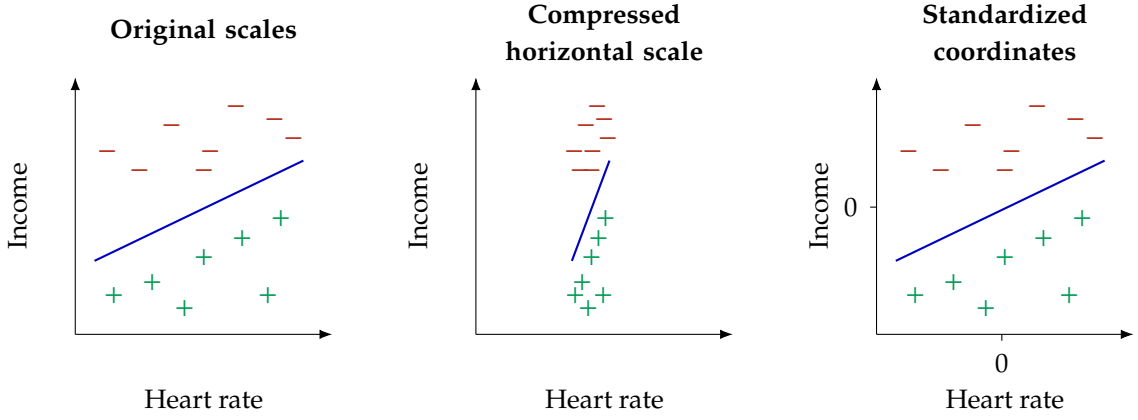


Figure 11: Schematic income and heart-rate geometry: changing the horizontal scale compresses the points and rotates the displayed boundary. Standardized coordinates put the features on comparable scales, with zero at each mean. The panels illustrate representations, not additional measurements.

Plotting also helps interpret features. In the hospital-indicator example, positives were seen at one hospital and negatives were not. This association alone does not establish the indicator's meaning for heart-disease prediction. Domain experts can help assess it.

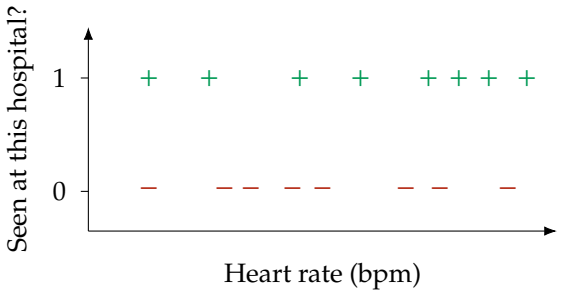


Figure 12: Schematic hospital indicator versus heart rate. The indicator alone separates these labels, making its meaning an essential part of interpretation.

Example: *The encoded observations.* Use the feature order

$$\phi(x) = (\text{heart rate}, \text{pain}, j_1, \ldots, j_5, m_1, m_2, \text{decade}, \text{income})^{\mathsf{T}}.$$

Using centering and scale pairs $(70, 10)$ for heart rate, $(3, 1)$ for decade, and $(50000, 40000)$ for income gives the following numerical representation. For example, $(55 - 70)/10 = -1.5$ and $(133000 - 50000)/40000 = 2.075$.

	y	Standardized heart rate	Pain	$(j_1, \ldots, j_5)$	(m_1, m_2)	Standardized decade	Standardized income
1	-1	-1.5	0	$(1,0,0,0,0)$	$(1,0)$	1	2.075
2	-1	0.1	0	$(0,1,0,0,0)$	$(1,1)$	-1	-0.4
3	$+1$	1.9	1	$(1,0,0,0,0)$	$(0,1)$	2	-0.25
4	-1	-0.3	0	$(0,0,0,1,0)$	$(0,0)$	2	1.75

Table 5: The transformed illustrative observations. The centering and scale constants specify this example; they are not sample statistics of these four rows.

3.6 Nonlinear boundaries from new features

A straight line cannot separate a negative cluster surrounded by positive points in the breathing-rate and blood-pressure example. A curved boundary can. More generally, classify by the sign of a score $f(x)$; its boundary is the set where $f(x) = 0$.

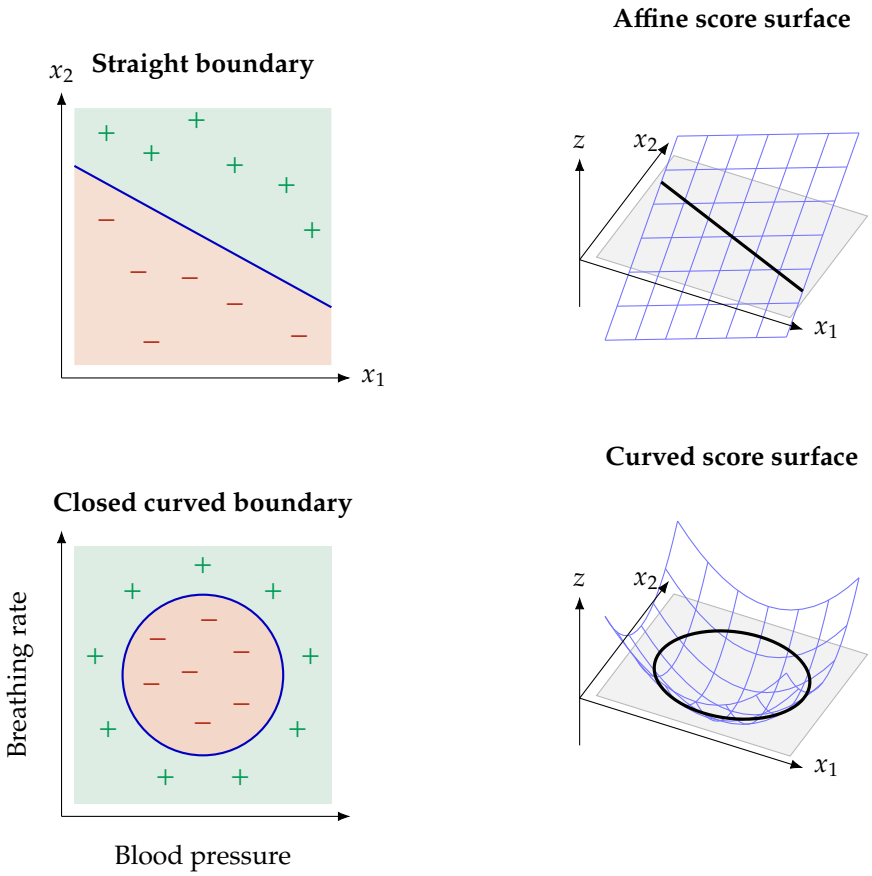


Figure 13: Schematic boundaries and score surfaces. An affine score gives a straight zero boundary; a bowl-shaped score gives a closed curved boundary. Intersecting a score surface with $z = 0$ identifies the classification boundary.

We can keep a linear classifier in the new coordinates while allowing a nonlinear function of the old coordinates:

$$f(x) = \theta^{\mathsf{T}}\phi(x) + \theta_0, \qquad h(x) = \begin{cases} +1, & f(x) > 0, \\ -1, & f(x) \le 0. \end{cases}$$

The expression is linear in $\phi(x)$ and its coefficients, but need not be linear in x. The choice of features therefore changes the available boundaries in the original input space.

3.7 Polynomial features and flexible boundaries

A sufficiently smooth function can be approximated locally by a Taylor polynomial, for example around 0. This motivates using polynomial terms as features. A polynomial of order k includes terms up to total degree k.

Order	Features for one input x_1	Features for d inputs
0	(1)	constant 1
1	$(1, x_1)$	constant and each x_j
2	$(1, x_1, x_1^2)$	previous terms and each $x_i x_j$, $i \le j$
3	$(1, x_1, x_1^2, x_1^3)$	previous terms and each $x_i x_j x_\ell$, $i \le j \le \ell$

Table 6: Polynomial feature vectors, shown as rows for readability. Mixed terms are included, not just powers of individual coordinates.

For order two, the added terms include $x_1^2, x_1x_2, \ldots, x_d^2$. For order three they also include $x_1^3, x_1^2x_2, x_1x_2x_3, \ldots, x_d^3$. A weighted sum of these features is a polynomial score. Its zero set can produce curved boundaries and more varied positive and negative regions.

Greater flexibility can also fit peculiarities of the training observations. In the heart-rate and weekly-exercise example, small positive regions around individual positive examples achieve zero training error. That alone does not show that the same rule will predict new observations well.

Definition: Overfitting occurs when a fitted rule captures training-specific detail in a way that does not carry over well to new data.

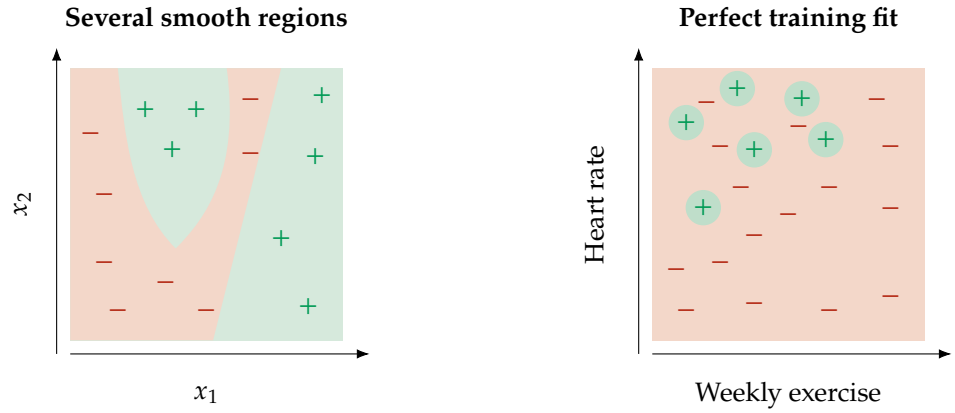


Figure 14: Schematic effects of flexible boundaries. The left panel shows several regions defined by a smooth score. On the right, small positive islands fit every training label in the heart-rate and exercise example. Zero training error leaves performance on new examples unresolved.

3.8 Evaluation with held-out data

Training error alone cannot assess predictions on new data.

A holdout split reserves data for testing. More training data better resembles fitting the full dataset; more test data gives a less noisy error estimate. Shuffle before splitting to remove ordering effects; one split may be unrepresentative.

3.9 Cross-validation

Cross-validation uses k roughly equal, nonempty chunks $\mathcal{D}_{n,1}, \ldots, \mathcal{D}_{n,k}$. Train h_i on all chunks except i and evaluate it on chunk i.

For a nonempty labeled set S, define its zero-one error by

$$\mathcal{E}(h, S) = \frac{1}{|S|} \sum_{(x,y) \in S} L(h(x), y).$$

The k-fold cross-validation error is

$$\mathcal{E}_{\mathrm{CV}} = \frac{1}{k} \sum_{i=1}^{k} \mathcal{E}(h_i, \mathcal{D}_{n,i}).$$

Each observation is held out once, with a fresh classifier trained on the other chunks.

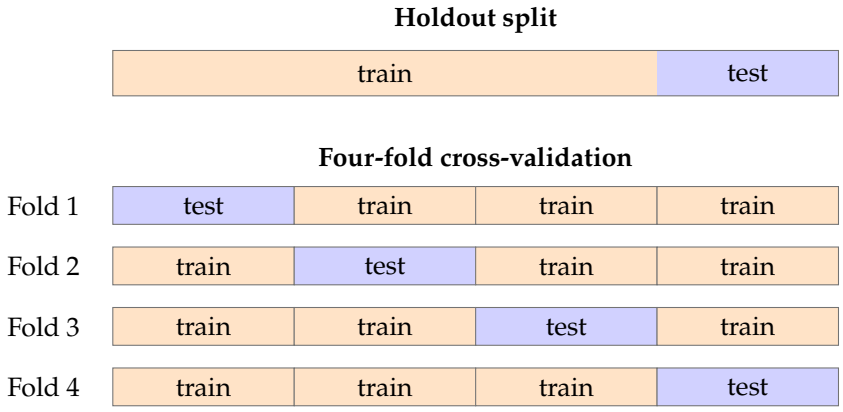


Figure 15: A schematic holdout split and four-fold cross-validation. Blue marks held-out data; pale orange marks training data. In each fold the blue chunk changes, and a fresh classifier is trained on the other chunks.

Algorithm 3.1: k**-fold cross-validation**

Input: Labeled data $\mathcal{D}_n$; learning procedure A; $2 \le k \le n$.
Output: Mean held-out error.

Shuffle the order of $\mathcal{D}_n$
Divide $\mathcal{D}_n$ into k nonempty, roughly equal chunks $\mathcal{D}_{n,1}, \ldots, \mathcal{D}_{n,k}$
for $i \leftarrow 1$ **to** k **do**
    $h_i \leftarrow A(\mathcal{D}_n \setminus \mathcal{D}_{n,i})$
    $e_i \leftarrow \mathcal{E}(h_i, \mathcal{D}_{n,i})$
return $\frac{1}{k} \sum_{i=1}^{k} e_i$

The returned quantity evaluates the learning procedure; it is not a single classifier. The formula gives equal weight to each fold error, even when chunk sizes differ slightly.

Source. Tamara Broderick, *Introduction to Machine Learning*, MIT 6.036/6.862, Lecture 3, Fall 2020.